

SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 1 OF 10

Python Internals & Core Concepts

19 Questions | Deep Technical Depth | Production-Grade Answers

Table of Contents

1. Difference between list and tuple
2. What is the GIL (Global Interpreter Lock)?
3. What are generators?
4. Difference between iterable and iterator
5. What is `__init__` and how does object creation really work?
6. What are decorators?
7. What are context managers?
8. What is monkey patching?
9. What is duck typing?
10. Mutable vs Immutable types
11. Deep copy vs Shallow copy
12. `is` vs `==`
13. Multithreading vs Multiprocessing
14. Asyncio vs threading
15. Memory management in Python
16. What happens during import?
17. How does virtualenv work?
18. Python sync vs async execution — what really happens?

Introduction

This document covers the Python Internals & Core Concepts you are expected to know deeply as a Senior Software Engineer. The questions here reflect what senior-to-principal level interviewers test — not just 'what is X' but 'how does X actually work and when does it matter in production'.

Approach: Read each answer as a refresher. The goal is to wire together the concepts you already know into crisp, precise language you can deliver confidently under interview pressure.

Q1: Difference between list and tuple

Both are ordered sequences, but the fundamental difference is mutability. A list is mutable — you can append, remove, or modify elements after creation. A tuple is immutable — once created, it cannot change.

```
lst = [1, 2, 3]
lst[0] = 99          # OK

tpl = (1, 2, 3)
tpl[0] = 99          # TypeError: 'tuple' object does not support item assignment
```

Memory: Tuples have a smaller memory footprint. CPython allocates exactly what's needed; lists over-allocate to amortize the cost of future appends.

Performance: Iterating a tuple is slightly faster than a list because Python can make optimization assumptions about immutable objects.

Hashability: Tuples containing only hashable elements are themselves hashable and can be used as dictionary keys or set members. Lists cannot.

```
d = {(1, 2): 'point'} # valid
d = {[1, 2]: 'point'} # TypeError: unhashable type: 'list'
```

Use case: Use tuples for heterogeneous, fixed-structure data (e.g., a (lat, lng) coordinate or a database row). Use lists for homogeneous, variable-length collections.

One subtle gotcha: a tuple containing a mutable object (like a list) is itself not deeply immutable — the tuple's reference to the list cannot change, but the list's contents can.

```
t = ([1, 2], 3)
t[0].append(99)    # Works! t is now ([1, 2, 99], 3)
t[0] = []          # TypeError – the reference is immutable
```

Q2: What is the GIL (Global Interpreter Lock)?

The GIL is a mutex in CPython that ensures only one thread executes Python bytecode at a time, even on multi-core systems. It exists because CPython's memory management — specifically reference counting — is not thread-safe without it.

Why it exists: Every Python object has a reference count. Without the GIL, two threads could simultaneously modify the same object's reference count leading to memory corruption or premature deallocation.

```
import sys
a = []
b = a
print(sys.getrefcount(a)) # 3 (a, b, getrefcount arg)
```

Impact on CPU-bound tasks: Threads do NOT give you true parallelism for CPU-bound work. Two threads computing Fibonacci will not run simultaneously on two cores — the GIL forces them to take turns.

Impact on IO-bound tasks: The GIL is released during IO operations (network, disk, sleep). So threads genuinely help with IO-bound tasks — while one thread waits for a response, another can run.

Workaround: For CPU parallelism, use multiprocessing — each process has its own GIL and its own interpreter. Or use C extensions that release the GIL (e.g., NumPy does this during computation).

Python 3.12 introduced experimental 'free-threaded' mode (PEP 703) to remove the GIL, but it is opt-in and still being battle-tested as of mid-2025.

Q3: What are generators?

A generator is a function that uses the `yield` keyword instead of `return`. Calling it returns a generator object — a lazy iterator that produces values one at a time, on demand, without storing the entire sequence in memory.

```
def count_up(n):
    i = 0
    while i < n:
        yield i    # suspends here, returns i to caller
        i += 1

gen = count_up(5)    # No code runs yet
print(next(gen))    # 0 — runs until first yield
print(next(gen))    # 1
list(count_up(5))    # [0, 1, 2, 3, 4]
```

Lazy evaluation: Generators compute values one at a time. Reading a 10GB log file line-by-line with a generator uses constant memory, not 10GB.

Generator expressions: Syntactic sugar: `(x*x for x in range(1000))` is a generator expression — like a list comprehension but lazy. Uses parentheses instead of brackets.

```
# List comprehension — builds full list in memory
squares_list = [x*x for x in range(1_000_000)]

# Generator expression — computes on demand
squares_gen = (x*x for x in range(1_000_000))
total = sum(squares_gen) # never stores 1M numbers
```

send() and two-way communication: Generators support `send(value)` — you can push a value back into the generator at the yield point. This is how coroutines worked before `async/await`.

Internals: Each generator object maintains its own stack frame — local variables, execution position, and state — so it truly suspends mid-function and resumes exactly where it left off.

Q4: Difference between iterable and iterator

These terms describe a protocol (interface), not a type. Understanding them precisely is important for writing Pythonic code.

Iterable: Any object that implements `__iter__()` which returns an iterator. Lists, tuples, dicts, sets, strings, and generators are all iterables. You can loop over them with `for`.

Iterator: Any object that implements both `__iter__()` (returns self) and `__next__()` (returns the next value or raises `StopIteration`). Generators and file objects are iterators.

```
lst = [1, 2, 3]        # iterable, NOT an iterator
it = iter(lst)         # iterator — has __next__
print(next(it))        # 1
print(next(it))        # 2

# A list can produce multiple independent iterators
```

```

it1 = iter(lst)
it2 = iter(lst)          # independent cursor

# An iterator can only go forward – it IS its own iterator
gen = (x for x in range(3))
print(iter(gen) is gen)  # True

```

Key distinction: A list is reusable — you can call `iter()` on it multiple times, each returning a fresh cursor. An iterator is consumed — once exhausted, calling `next()` always raises `StopIteration`; `iter()` returns the same exhausted object.

Q5: What is `__init__` and how does object creation really work?

`__init__` is the initializer, not the constructor. The actual constructor is `__new__`, which allocates and returns the new object. `__init__` then receives that already-created object (`self`) and sets its initial state.

```

class Foo:
    def __new__(cls, *args, **kwargs):
        print('__new__ called – allocating object')
        instance = super().__new__(cls)
        return instance

    def __init__(self, value):
        print('__init__ called – initializing')
        self.value = value

f = Foo(42)
# Output:
# __new__ called – allocating object
# __init__ called – initializing

```

When to override `__new__`: Singletons, immutable types (like subclassing `int` or `str`), and metaclass patterns. For normal classes, `__init__` is sufficient.

Instance dict: Unless `__slots__` is used, every instance gets a `__dict__` storing its attributes. `__init__` typically just populates this dict.

Q6: What are decorators?

A decorator is a function that takes a function as input and returns a modified or wrapped function. It is syntactic sugar for higher-order function application.

```

def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before call')
        result = func(*args, **kwargs)
        print('After call')
        return result
    return wrapper

@my_decorator
def greet(name):
    print(f'Hello, {name}!')

greet('Gautham')          # Before call → Hello, Gautham → After call

```

`functools.wraps`: Without it, `greet.__name__` would be `'wrapper'`. Always apply `@functools.wraps(func)` inside your wrapper to preserve the original function's metadata.

Parameterized decorators: Add another layer — a factory function that returns a decorator. e.g., `@retry(times=3)` is a function call that returns the actual decorator.

```
def retry(times=3):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try: return func(*args, **kwargs)
                except Exception: pass
            return wrapper
        return decorator

    @retry(times=5)
    def call_api(): ...
```

Class decorators: Decorators can also be classes that implement `__call__`. FastAPI, Flask, and LangChain heavily use both function and class decorators to register routes and tools.

Q7: What are context managers?

A context manager is any object that implements `__enter__` and `__exit__`. The `with` statement calls `__enter__` on entry and guarantees `__exit__` is called on exit, even if an exception is raised — making it the idiomatic way to manage resources.

```
with open('file.txt', 'r') as f:
    data = f.read()
# file is always closed here – even if read() raised

# Equivalent to:
f = open('file.txt', 'r')
try:
    data = f.read()
finally:
    f.close()
```

`__exit__` signature: `__exit__(self, exc_type, exc_val, exc_tb)` — if no exception, all three are `None`. Return `True` to suppress the exception; return `None/False` to propagate it.

`contextlib.contextmanager`: A decorator that turns a generator function into a context manager. Everything before `yield` is `__enter__`; everything after `yield` is `__exit__`.

```
from contextlib import contextmanager

@contextmanager
def timer():
    start = time.time()
    try:
        yield # 'as' value would come from here
    finally:
        print(f'Elapsed: {time.time() - start:.3f}s')

with timer():
    expensive_operation()
```

In async code, use `async with` and implement `__aenter__` / `__aexit__` — or use `asynccontextmanager` from `contextlib`. SQLAlchemy sessions, httpx clients, and async DB connections all use this pattern.

Q8: What is monkey patching?

Monkey patching is dynamically replacing or extending attributes, methods, or modules at runtime without modifying the original source code.

```
import json

original_dumps = json.dumps

def patched_dumps(obj, **kwargs):
    print('json.dumps called!')
    return original_dumps(obj, **kwargs)

json.dumps = patched_dumps    # monkey patch
json.dumps({'key': 'val'})    # prints the log
```

Legitimate uses: Test mocking (unittest.mock), hot-fixing third-party libraries with bugs, adding behavior to classes you don't own (gevent uses this to make blocking stdlib calls async).

Risks: Hidden side effects, breakage on library upgrades, difficult debugging, and non-obvious code behavior. Use sparingly and document clearly.

Q9: What is duck typing?

'If it walks like a duck and quacks like a duck, it's a duck.' Python cares about what an object can do, not what type it is. You never need to check isinstance before calling a method — you just call it.

```
def process(file_like):
    return file_like.read()    # works for file, BytesIO, StringIO, any .read()

process(open('log.txt'))      # real file
process(io.StringIO('hello')) # in-memory string
process(requests.get(url).raw) # HTTP response stream
```

EAFP vs LBYL: Python favors EAFP (Easier to Ask Forgiveness than Permission): try to use an object and handle exceptions, rather than checking types upfront (Look Before You Leap). Duck typing enables EAFP.

Protocols (modern duck typing): Python 3.8+ introduced Protocols in typing — structural subtyping that formalizes duck typing. A class satisfies a Protocol if it has the required methods/attributes, without explicitly inheriting.

Q10: Mutable vs Immutable types

Mutable objects can be changed after creation; immutable objects cannot. This distinction affects identity, hashability, default argument traps, and concurrency safety.

Immutable types: int, float, bool, str, bytes, tuple, frozenset. When you 'modify' a string, Python creates a new string object.

Mutable types: list, dict, set, bytearray. Modifications happen in-place on the same object.

```
# The famous mutable default argument trap
def add_item(item, lst=[]):
    lst.append(item)
    return lst

add_item(1)    # [1]
add_item(2)    # [1, 2] ← NOT [2] – default list is shared across calls!

# Fix: use None sentinel
def add_item(item, lst=None):
    if lst is None: lst = []
```

```
lst.append(item)
return lst
```

Identity vs equality: Two immutable values with the same content may share the same object (interning). CPython interns small integers (-5 to 256) and some strings. This means ``a is b`` can be True for two 'different' variable assignments.

Q11: Deep copy vs Shallow copy

When you copy an object, the question is: are nested mutable objects shared or duplicated?

Shallow copy: Creates a new container but does NOT recursively copy nested objects — inner objects are shared references. ``list.copy()``, ``dict.copy()``, and ``copy.copy()`` all produce shallow copies.

Deep copy: ``copy.deepcopy()`` recursively copies every object, producing a fully independent clone. Much slower and memory-intensive, but safe when you need complete isolation.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
shallow[0].append(99)
print(original)  # [[1, 2, 99], [3, 4]] – inner list is shared!

deep = copy.deepcopy(original)
deep[0].append(88)
print(original)  # [[1, 2, 99], [3, 4]] – unchanged
```

Practical implication: When passing mutable objects to functions or between services, be explicit about whether you want sharing or isolation. In FastAPI Pydantic models, `.model_copy()` handles this cleanly.

Q12: is vs ==

==: Equality check — calls `__eq__`. Tests whether two objects have the same value.

is: Identity check — compares memory addresses (`id()`). Tests whether two names point to the exact same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b    # True – same value
a is b    # False – different objects in memory

# CPython integer interning
x = 256
y = 256
x is y    # True – CPython caches -5 to 256

x = 257
y = 257
x is y    # False (in most implementations) – not interned

# Correct: always use 'is' for None/True/False checks
if result is None: ...
if result == None: ... # works but fragile – __eq__ can be overridden
```

Q13: Multithreading vs Multiprocessing

Both provide concurrency, but they differ fundamentally in memory model and when they actually achieve parallelism in Python.

Multithreading: Multiple threads share the same memory space. In CPython, the GIL prevents true parallel execution of Python bytecode. Good for IO-bound tasks; wastes CPU for compute-heavy work.

Multiprocessing: Multiple separate processes, each with its own interpreter and GIL. True CPU parallelism. Communication requires IPC (queues, pipes, shared memory). Higher overhead per process.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# IO-bound: fetching 100 URLs – threads win
with ThreadPoolExecutor(max_workers=20) as ex:
    results = list(ex.map(fetch_url, urls))

# CPU-bound: image compression – processes win
with ProcessPoolExecutor(max_workers=8) as ex:
    results = list(ex.map(compress_image, images))
```

Thread safety: With threads, shared mutable state requires locks (threading.Lock). With processes, you're isolated by default but serialization (pickle) has overhead.

Practical rule: IO-bound → threads or asyncio. CPU-bound → multiprocessing or C extensions that release the GIL.

Q14: Asyncio vs threading

Both solve IO-bound concurrency, but through different mechanisms — cooperative vs preemptive multitasking.

Threading: OS-managed, preemptive. The OS decides when to switch threads. Each thread has its own stack. Easier to retrofit existing blocking code. Unpredictable switching requires locks.

Asyncio: Single-threaded, cooperative. You explicitly yield control at `await` points. No locking needed for shared state (single thread). Much lower memory — thousands of coroutines vs. threads (each thread ~8MB stack).

```
# Threading – each request blocks its thread during IO
def handle_request(req):
    data = db.query(req.id) # blocks thread
    return data

# Asyncio – single thread handles all requests
async def handle_request(req):
    data = await db.query(req.id) # yields control during IO
    return data
```

When asyncio wins: High-concurrency servers with many simultaneous IO operations. FastAPI, aiohttp, and modern async ORMs (SQLAlchemy 2.0, Tortoise ORM) are built around this model.

When threading still makes sense: Integrating with blocking legacy libraries that you can't make async. ThreadPoolExecutor to run blocking code without blocking the event loop.

Q15: Memory management in Python

Python manages memory through reference counting as the primary mechanism, with a cyclic garbage collector as a fallback.

Reference counting: Every object has a refcount. When it hits 0, memory is immediately freed. Fast and deterministic — no pause for GC.

Cyclic GC: Reference counting fails for cycles ($A \rightarrow B \rightarrow A$). Python's gc module detects and collects these. Runs periodically; can be triggered with gc.collect().

```
import sys, gc

a = [1, 2, 3]
print(sys.getrefcount(a))    # 2 (a + getrefcount arg)
b = a
print(sys.getrefcount(a))    # 3
del b
print(sys.getrefcount(a))    # 2 again

# Cycle — neither a nor b reaches refcount 0
a = {}
b = {'ref': a}
a['ref'] = b
del a, b    # still alive — requires cyclic GC
```

Memory pools: CPython uses its own allocator (pymalloc) for small objects (< 512 bytes) to avoid the overhead of calling malloc/free for every tiny object.

__slots__: By default, each instance has a __dict__. Defining __slots__ replaces the dict with a fixed-size array — less memory and faster attribute access. Important for classes instantiated millions of times.

Q16: What happens during import?

Python's import system is more complex than it appears. Understanding it explains module caching, circular imports, and import hooks.

```
# Step-by-step when Python executes: import mymodule

# 1. Check sys.modules cache (most imports hit here)
# 2. Find the module: search sys.path in order
#    (current dir, PYTHONPATH, standard lib, site-packages)
# 3. Load: compile source to bytecode (.pyc), or load from cache
# 4. Execute: run the module's top-level code
# 5. Cache: store in sys.modules
# 6. Bind: the name 'mymodule' in the current namespace
```

sys.modules: The module registry. Once imported, every subsequent `import mymodule` is a dict lookup — no file I/O, no re-execution. This is why mutable module-level state persists across imports.

Circular imports: Module A imports B, which imports A — the partial module object is in sys.modules but may not have the attributes the caller needs yet. Fix by deferring imports inside functions or restructuring.

__pycache__: Python compiles .py to .pyc (bytecode) and caches it in __pycache__. On subsequent imports, if the .py hasn't changed, it loads .pyc directly — faster but same semantics.

Q17: How does virtualenv work?

A virtualenv is an isolated Python environment with its own site-packages, and optionally its own Python binary. It solves the dependency isolation problem between projects.

Mechanism: It creates a directory with a Python binary (or symlink to system Python) and its own lib/site-packages. The `activate` script prepends the venv's bin to PATH and sets VIRTUAL_ENV.

```
python -m venv .venv    # Create venv
source .venv/bin/activate    # Activate (modifies PATH, PS1)
```

```
which python          # .venv/bin/python
pip install fastapi    # installs to .venv/lib/python3.x/site-packages
deactivate            # restore PATH
```

pip internally: pip fetches packages from PyPI (or a configured index), resolves dependencies, downloads .whl (wheel) files, and extracts them to site-packages. pip install reads setup.py or pyproject.toml for metadata.

Modern tooling: uv (by Astral) is a Rust-based replacement for pip and virtualenv — 10-100x faster. Poetry and PDM manage both venvs and lockfiles. In production containers, you typically just install directly without activating a venv.

Q18: Python sync vs async execution — what really happens?

In synchronous Python, each statement blocks until complete. The call stack is straightforward. In async Python, the event loop is a scheduler that multiplexes many coroutines on a single thread.

```
# Sync — each line blocks
def fetch_data():
    result = requests.get(url)  # blocks entire thread for 200ms
    return result.json()

# Async — yields control at await
async def fetch_data():
    async with httpx.AsyncClient() as client:
        result = await client.get(url)  # event loop runs other coroutines
    return result.json()

# Running async code
asyncio.run(fetch_data())  # creates event loop, runs coroutine, closes
```

Key insight: async/await does NOT add threads or processes. It is single-threaded cooperative multitasking. The benefit is that while waiting on IO (network, disk), other coroutines can run instead of the CPU sitting idle.

Mixing sync and async: Never call a blocking function (time.sleep, requests.get, a synchronous ORM) inside an async function. It blocks the event loop, freezing ALL other coroutines. Use asyncio.to_thread() to run blocking code in a thread pool without blocking the loop.
